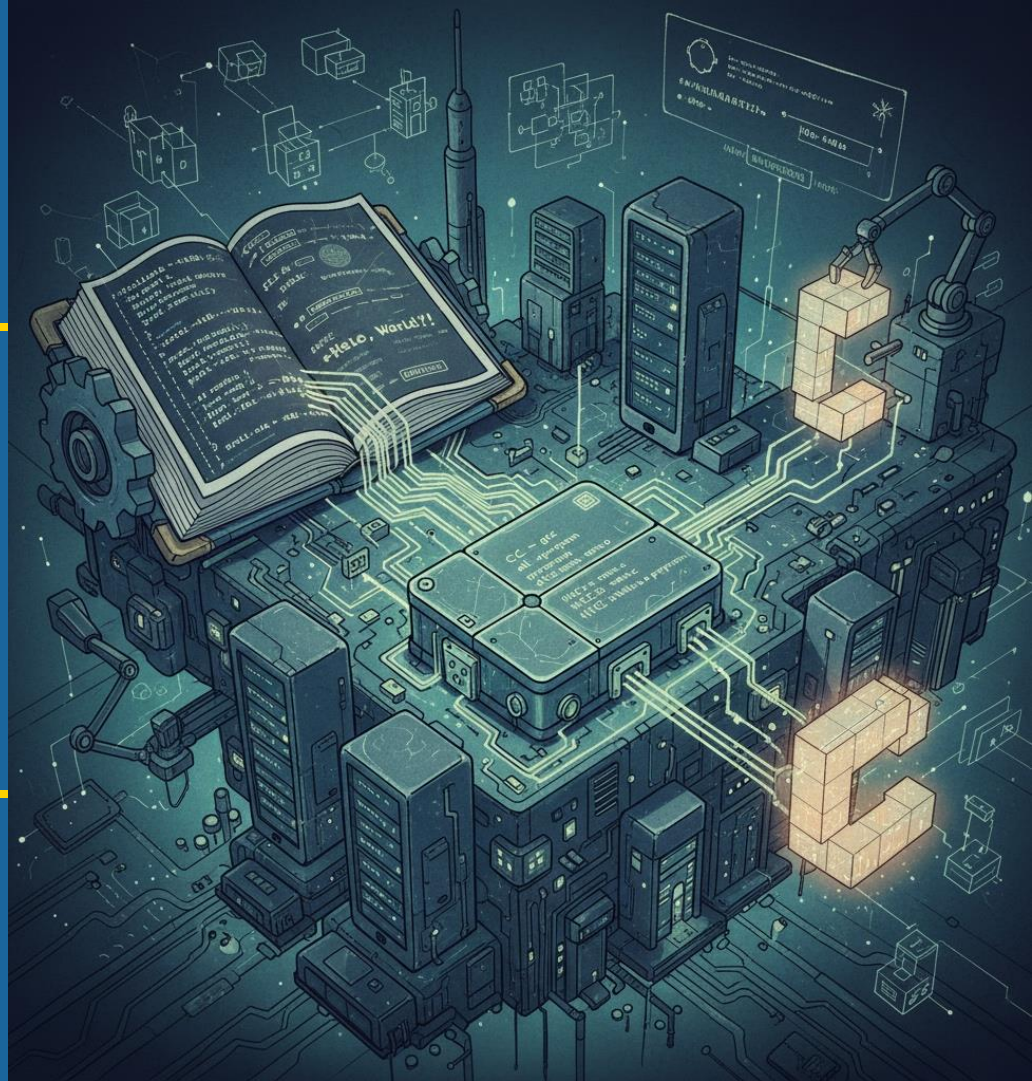
UCLA Samueli School of Engineering

CS 35L Software Construction

Lecture 13 – Make & C Programming

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



Why Learn C?

Speed

C **compiles directly to machine code** & is very **close to direct hardware instructions.** This allows experts to **optimize performance**

Memory Management

C allows for explicit and direct manipulation of **memory.**

This requires **careful coding.** It enables programmers to write **highly optimized code and manage resources precisely**.

Hardware Interaction

C is unmatched when you need to interface directly with hardware, making it essential for **device drivers** and **firmware**.

Most of the Software you use Every Day is Runs on Applications Written in C

Operating Systems
Linux & Windows kernels due to speed & direct access to hardware & memory

Embedded Systems and IoT devices
due to small memory footprint and direct hardware access

Compilers and **Assemblers**
due to small memory footprint and direct hardware access

Database Management Systems
MySQL, PostgreSQL core code

C++ builds on C

- C is purely **procedural** (functions and data are kept separate) while C++ is object-oriented (encapsulating functions and data into classes)
- **No Classes** or **Objects**
  - No inheritance or polymorphism
  - Structs allow you to define data structures

```
struct list_element {
        int value;
        struct list_element* next; // Linked list
};
```

C has No Function Overloading

Each function must have a **unique name**

In C++:

```
void print(int value)
{…}

void print(float value)
{…}

int main() {
    int a = 5;
    float b = 5.00;
    print(a);
    print(b);
}
```

Function Overloading (functions are allowed to have the same name but different signature)

In C:

```
void printInt(int value)
{…}

void printFloat(float value)
{…}

int main() {
    int a = 5;
    float b = 5.00;
    printInt(a);
    printFloat(b);
}
```

C has No Pass-by-References

- Only pointers are available
- Parameters passing always involved either values or pointers

In C++:

```
void swap(int &a, int &b) {
  int temp = a;
  a = b;
  b = temp;
}

int main() {
  int x = 30;
  int y = 40;
  swap(x, y);
```

Pass by Reference
passing arguments to a function where the function receives a reference to the original variable in memory, rather than a copy of its value

Pure Pointers

In C:

```
void swap(int *a, int *b) {
  int temp = *a;
  *a = *b;
  *b = temp;
}

int main() {
  int x = 30;
  int y = 40;
  swap(&x, &y);
```

No built-in try/catch blocks for Error Handling

- Instead: return values are used to indicate error

In C++:

```
int safe_divide(int num, int den) {
  if (num == 0) {
    throw std::runtime_error("div by 0");
  }
  return numerator / denominator;
}
int main() {
  int x = 10, y = 0, z;
  try {
    z = safe_divide(x, y);
    printf("Result: %d\n", z);
  }
  catch (const std::runtime_error& e) {
    std::cerr << " Error: Division by zero
occurred " << e.what() << std::endl;
  }
}
```

Now the result has to be a pointer argument

In C:

```
int safe_divide(int num, int den, int* result) {
  if (num == 0) {
    return -1; // Indicate an error
  }
  *result = num / den;
  return 0; // Indicate success
}
int main() {
  int x = 10, y = 0, z;
  if (safe_divide(x, y, &z) != 0) {
    printf("Error: Division by zero occurred.\n");
  } else {
    printf("Result: %d\n", z);
  }
  return 0;
}
```

Again, WWHHYYYY Would Someone Use C Instead of C++ ???

Smaller Executable

C compilers often generate **smaller binaries** because C doesn't include the large runtime support libraries or necessary metadata associated with C++ features like exceptions or virtual tables & constructors/destructors for objects. This is crucial for environments with **limited memory** (e.g., embedded systems)

Predictable Execution

C gives the developer **almost complete control over memory allocation and function calls**. C++ introduces hidden operations (like constructor calls, exception unwinding, and virtual function lookups) that can **make execution time less predictable.**

Again, WWHHYYYY Would Someone Use C Instead of C++ ???

Library Interface

Almost all programming languages (including Python, Java, C#) support **calling C functions** & can therefore **use C libraries.**

Picking C allows you to write a library with the **widest possible compatibility**

```
import ctypes
import os

# 1. Determine the correct library
# file name based on the OS
if os.name == 'posix': # Linux or macOS
    lib_file = './my_c_lib.so'
elif os.name == 'nt':  # Windows
    lib_file = './my_c_lib.dll'
else:
    raise OSError("Unsupported OS.")
```

```
# 2. Load the shared library
try:
    c_lib = ctypes.CDLL(lib_file)
except OSError as e:
    print(f"Error loading library: {e}")
    print("Make sure you compiled the C
file into the correct shared library.")
    exit()
```

Memory Management in C

- `void *malloc(size_t size)` allocates memory of a given size on the heap
- `void free(void *ptr)` deallocates the memory
  - If you don't free the memory, you will have a memory leak
  - Accessing memory that is not currently allowed by your program results in a segmentation fault ("segfault")

```
int* matrix1 = (int*)malloc(rows * cols * sizeof(int));
[…]
free(matrix1);
```

See https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/

Reading a File in C

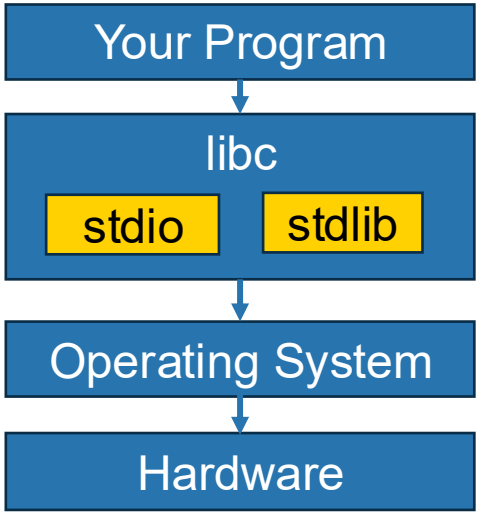
- `fopen` opens a file
- `fread` reads a given amount of data from a file stream
- `fclose` closes the file (you should always close the file after you are done reading)
- All three are *library calls*

```c
#include <stdio.h>
int main() {
    FILE *file;
    int buffer[5];

    // Open the binary file for reading
    file = fopen("input.bin", "rb");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }

    // Read the integers from the file into the buffer
    fread(buffer, sizeof(int), 5, file);

    for (int i = 0; i < 5; i++) {
        printf("Element %d: %d\n", i + 1, buffer[i]);
    }

    // Close the file
    fclose(file);
    return 0;
}
```

See: https://www.geeksforgeeks.org/c/fread-function-in-c/

Library Calls

- **`fread`** is a standard C library function that provides buffered input/output operations
- While **`fread`** interacts with the operating system to read data from files, it does so by making calls to lower-level system calls as needed.
- **`malloc`** and **`free`** are also library calls
- When creating an executable, the **linker** selects the **`libc`** implementation for your target operating system



Compiler & Linker Create an Executable from Source Code

Compiler/Assembler

Translates your source code into assembly code (also knows as an “object file”)

Linker

Resolves referenced but not defined symbols (function names, global variables)

Copies code sections from object files that define those symbols into the executable (**static linking**) or defers until program execution (**dynamic linking**)

Important C Language Elements

- **Characters & Strings**:
  - `'a'` is a single character (`'\0'` not the character for zero but the character with ASCII value zero (end of string)
  - `"a"` is a string **(`char* or char[]`)** (don't forget **`#include <string.h>`**)
- Use **`const`** to indicate that the variable should not be changed
  - **`const char*`** does not let you write into the memory of the pointer
  - **`const`** in C is a great way to avoid accidentally usings a variable in the wrong way so it prevents some bugs
  - you can cast away the costness but avoid this!)

```
char buffer[] = "Initial string"; // Modifiable array on the stack
const char *p_const = buffer; // p_const prevents modifications to the buffer
char *p_writable = (char *)p_const; // p_writable re-enables modifications
```

Goto allows you to jump to a different, Labeled Code Location

```c
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num > 0) {
        goto positive;
    }
    goto end;

positive: // A label named 'positive'
    // This block is reached only by the goto statement above
    printf("It is a positive number.\n");

end: // A label named 'end' for a clean exit point

    printf("Program finished.\n");
    return 0;
}
```

Edsger Dijkstra: "Go To Statement Considered Harmful"

```c
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num > 0) {
        goto positive;
    }
    goto end;

positive: // A label named 'positive'
    // This block is reached only by the go
    printf("It is a positive number.\n");

end: // A label named 'end' for a clean exi

    printf("Program finished.\n");
    return 0;
}
```

Don't use GOTO!

Edgar Dijkstra: Go To Statement Considered Harmful

See https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing
CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming languages (i.e. everything except, perhaps, plain machine code). At that time I did not attach too much importance to this discovery; I now submit my considerations for publication because in very recent discussions in which the subject turned up, I have been urged to do so.

My first remark is that, although the programmer's activity ends when he has constructed a correct program, the process taking place under control of his program is the true subject matter of his activity, for it is this process that has to accomplish the desired effect; it is this process that in its dynamic behavior has to satisfy the desired specifications. Yet, once the program has been made, the "making" of the corresponding process is delegated to the machine.

My second remark is that our intellectual powers are rather

dynamic progress is only
call of the procedure w
we can characterize the
textual indices, the ler
dynamic depth of proce

Let us now consider
or **repeat** *A* **until** *B*).
superfluous, because w
recursive procedures. F
clude them: on the on
mented quite comfortal
the other hand, the re
makes us well equippe
processes generated by
the repetition clauses
describe the dynamic pr
a repetition clause, ho
namic index," inexora
corresponding current
procedure calls) may b
progress of the process
(mixed) sequence of te

The main point is tl
programmer's control;
of his program or by the
he wishes or not. They

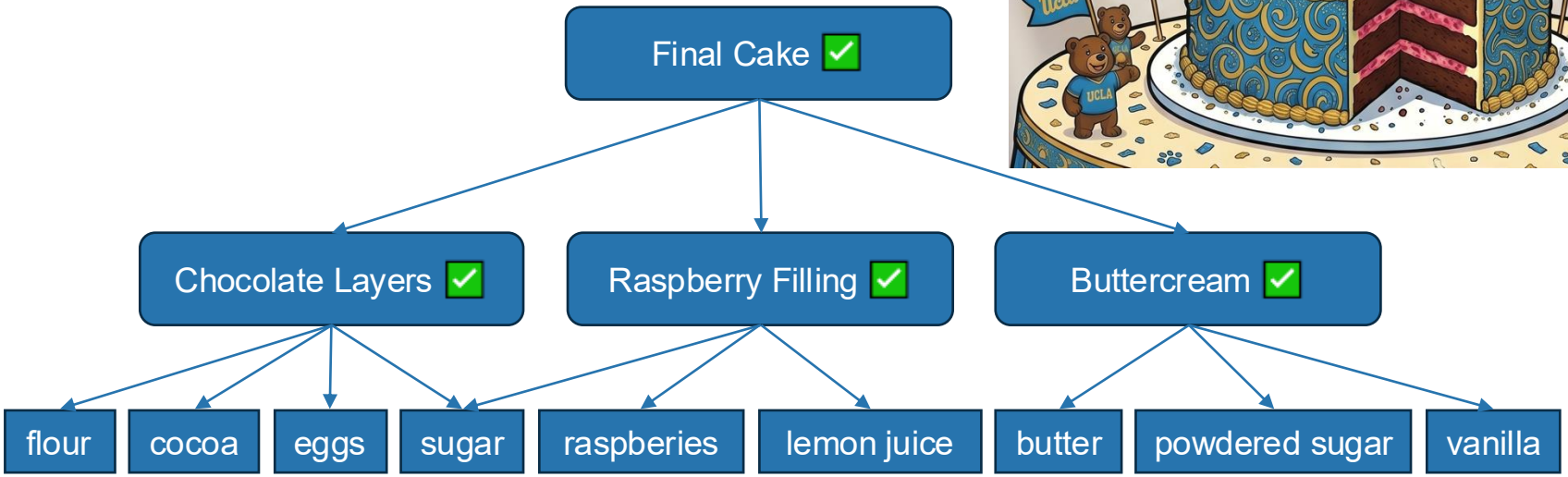
UCLA Samueli School of Engineering

CS 35L Software Construction

Lecture 13 – Make & Makefiles

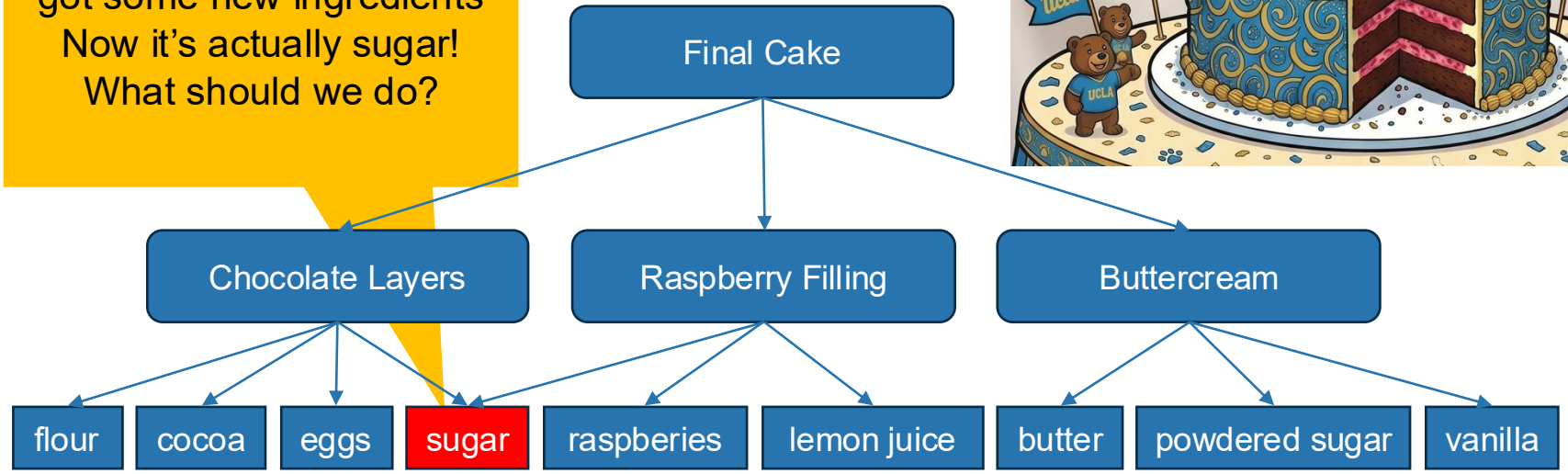
Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department

Let's Bake a Bruin Cake!

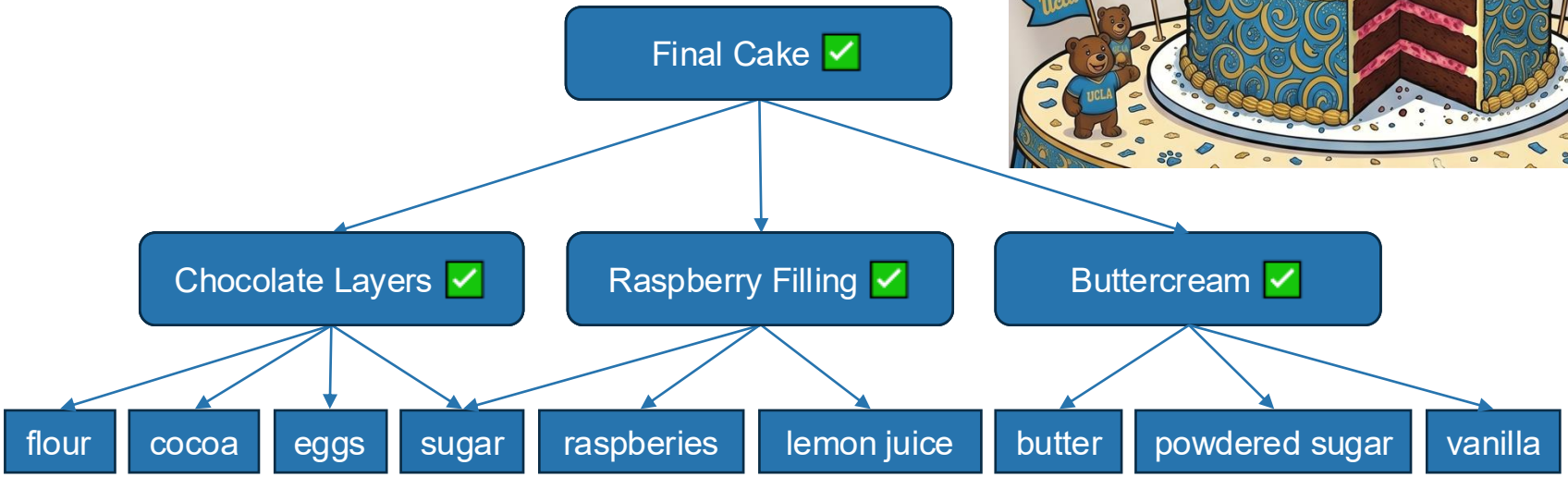


Let's Bake a Bruin Cake!

Was actually SALT! But we got some new ingredients
Now it's actually sugar!
What should we do?



Let's Bake a Bruin Cake!



`GNU Make` is a Universal Build Automation Tool

Goal: Incremental Build Automation

With so many **different source files**, **compilation & linking steps**, how can I **automate the build** of a large program while **only re-building** parts of the program for which the source files **have changed**?

Solution: GNU Make

- Automates **building, installing, uninstalling, and testing a package**
- Figures out automatically which **files it needs to update**, based on which source files have changed.
- **Language-independent** (C, C++, Java, LaTeX, …)**]**
- Specify steps in **`Makefile`**

See https://www.gnu.org/software/make/

See https://makefiletutorial.com/

Anatomy of a Very Simple Makefile

Build Target

A **product / file** you want to produce (e.g., executable, object file, …)
or a **task** you want to perform (e.g., “install”, “test”).
The name of the build target should be the **name of the produced file**.

Prerequisites

A **list of build targets** that need to be made before this build target and **files** that are used to make this target

Makefile

```
target: prereq_1 prereq_2 …
        command_1
        command_2

mysrc.o: mysrc.c

        cc mysrc.c -o mysrc.o
```

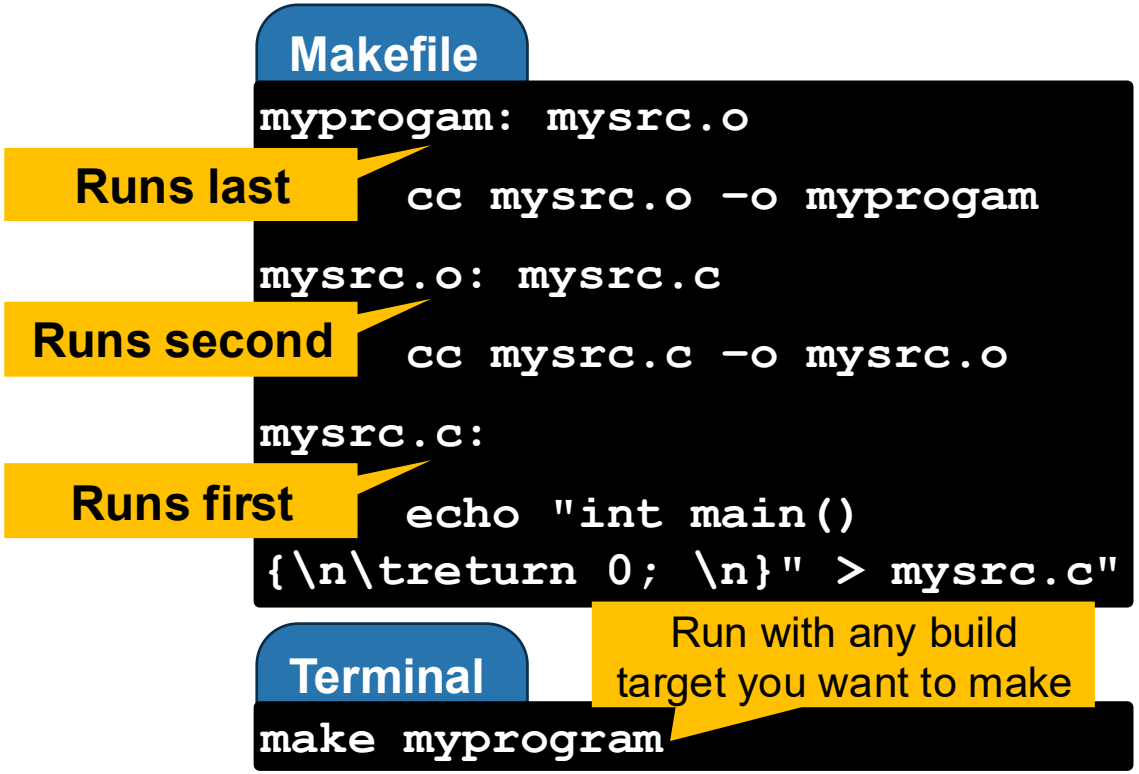
`cc` **is the C compiler**

Command

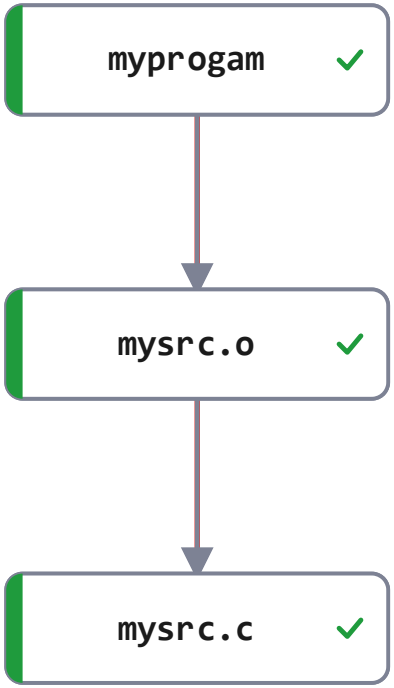
a sequence of **shell commands** to build the target or to perform the task

Make Allows you to Build in Multiple Steps

- Before making a build target, make **first makes all the dependencies** (unless they have not changed)
- Dependencies have **changed** if and only if the **modification date** of their file is **newer** than that of the build target



Make Allows you to Build in Multiple Steps



Makefile

```
myprogam: mysrc.o
	cc mysrc.o -o myprogam
mysrc.o: mysrc.c
	cc mysrc.c -o mysrc.o
mysrc.c:
	echo "int main()
{\n\treturn 0; \n}" > mysrc.c"
```

Runs last

Runs second

Runs first

Terminal

```
make myprogram
```

Run with any build target you want to make

If a Build Target should ALWAYS be made, Define it as .PHONY

See https://makefiletutorial.com/

How it works

- Commands of build targets listed as `.PHONY` will always be executed when the build target is made.

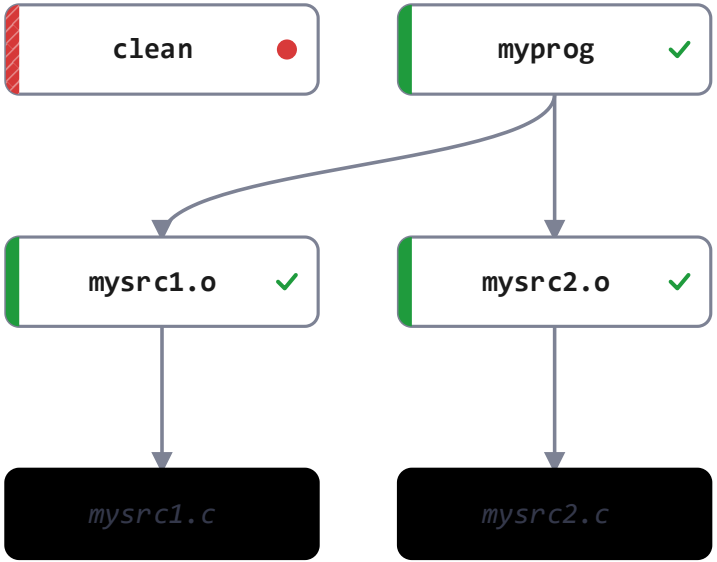
Makefile

```
.PHONY: test clean
test:
	pytest tests
clean:
	rm -r *.o
```

When to use Phony

- You want to **create a file with the name of a build target** that a task that does not make this files (e.g., “test”)
- Performance: Skip checking whether the files are new
- Commands commonly made phony: all, clean, test

Makefiles Support Variables



Makefile

```
VARIABLE_NAME = definition
SRCS = mysrc1.c mysrc2.c
OBJS = $(SRCS:.c=.o)
myprog: $(OBJS)
	cc $(OBJS) -o myprog
```

Makefiles Support Variables

- Variables are defined with a string that should substitute the variable name later
- `$(VARNAME)` uses the variable
- Make supports string-replace in variables using the syntax: `$(VAR:search_pattern=replacement_text)`

Makefile

```
VARIABLE_NAME = definition
SRCS = mysrc1.c mysrc2.c
OBJS = $(SRCS:.c=.o)
myprog: $(OBJS)
	cc $(OBJS) -o myprog
```

Replaces .c with .o

Translates to:

```
cc mysrc1.o mysrc2.o -o myprog
```

See https://makefiletutorial.com/

Pattern Rules are Templates For Creating Many Rules with the Same Structure

- `%` is a wildcard that matches any non-empty substring
- When used in the prerequisite pattern, the wildcard `%` adds the matched string.
- `$@` is the name of the build target of the rule
- `$<` is the first prerequisite of the rule (`$^` is all prerequisites)

Makefile

Can we use variables to refactor the Makefile?

```
VARIABLE_NAME = definition

SRCS = mysrc1.c mysrc2.c

OBJS = $(SRCS:.c=.o)

myprog: $(OBJS)
	cc $(OBJS) -o myprog

%.o: %.c
	cc $< -o $@
```

For mysrc1.o translates to:

```
cc mysrc1.c -o mysrc1.o
```

Pattern Rules are Templates For Creating Many Rules with the Same Structure

See https://makefiletutorial.com/

Variables allow you to **refactor** your Makefile to **reduce duplication.**
If we want to rename the program name, we only need to do this in one place

Can we refactor this Makefile even further???

Makefile

```
SRCS = mysrc1.c mysrc2.c

TARGET = myprog

OBJS = $(SRCS:.c=.o)

$(TARGET): $(OBJS)
	cc $(OBJS) -o $(TARGET)

%.o: %.c
	cc $< -o $@

clean:
	rm -f $(OBJS) $(TARGET)
```

Pattern Rules are Templates For Creating Many Rules with the Same Structure

See https://makefiletutorial.com/

If we want to use a different C compiler (gcc, clang) we can simply change this here

Makefile

```
SRCS = mysrc1.c mysrc2.c
TARGET = myprog
OBJS = $(SRCS:.c=.o)
CC = gcc
$(TARGET): $(OBJS)
	$(CC) $(OBJS) -o $(TARGET)
%.o: %.c
	$(CC) $< -o $@
clean:
	rm -f $(OBJS) $(TARGET)
```

The stereotypical Makefile for C Programs

If we want to compile with **different compiler flags** (arguments to the compiler) we can change this here

Runs static analysis (automatic analysis of your source code) to detect issue such as resource leaks due to missing `free` and `fclose`, double `free`, null pointer dereferences

Makefile

```
SRCS = mysrc1.c mysrc2.c
TARGET = myprog
OBJS = $(SRCS:.c=.o)
CC = clang
CCFLAGS = -Wall
CHECKFLAGS = --analyze -Xanalyzer
                        -analyzer-checker=core

$(TARGET): $(OBJS)
	$(CC) $(CCFLAGS) -o $(TARGET) $(OBJS)
%.o: %.c
	$(CC) $(CCFLAGS) -c $< -o $@
clean:
	rm -f $(OBJS) $(TARGET)
check:
	$(CC) $(CCFLAGS) $(CHECKFLAGS) $(SRCS)
```

Shows all compiler warnings

Please fill out your Exit Tickets on Bruin Learn!

Question 1 — 1 pts

In your own words, please summarize **three key insights** you learned about **C Programming and Makesfiles** today. (Do not copy phrases from the lecture slides)

Question 2 — 1 pts

Why is GNU Make **more useful** for C/C++ than for Python and JavaScript? Please describe **one use case** that applies to C/C++ but **not to Python & JS** and **one use case** that **still applies to Python & JS.**

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slide use images from Flaticon.com (Creators: Freepik)